

||Jai Sri Gurudev||

Sri Adichunchanagiri Shikshana Trust ®

BGS NATIONAL PUBLIC SCHOOL

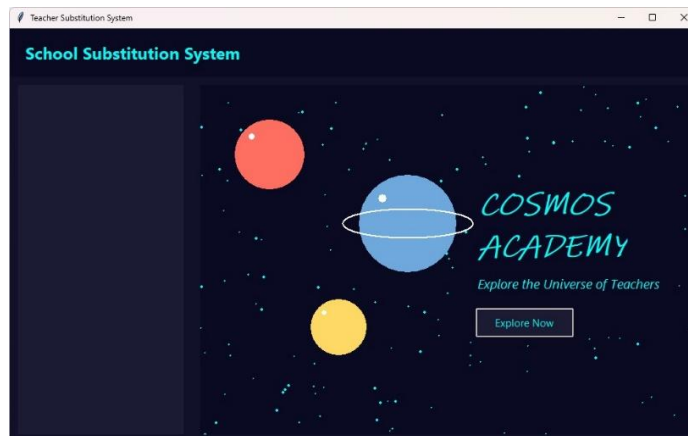
(Affiliated to Central Board of Secondary Education, New Delhi)
Hulimavu, Bannerghatta Road, Bengaluru - 560 076



Computer Science Investigatory Project

Year: 2025-26

Topic: Teacher Substitution



Name: S Lakshana

Class: 12 A

Exam Roll No:

CERTIFICATE

This is to certify that **S LAKSHANA** of class XII of BGS National Public School has successfully completed the Investigatory Project in Computer Science for **SENIOR SECONDARY CERTIFICATE EXAMINATION (SSCE)** prescribed by CBSE in the year 2025-2026.

Date:

Principal Sign

External Examiner

Internal Examiner

ACKNOWLEDGEMENTS

I would like to express my gratitude to our Respected **Principal Dr Malini M Dutta, and Vice Principal Ms Savitha Suverna** for being a constant pillar of support.

I would then like to thank our **Computer Science teacher, Ms. B. Ramya Shenoy** for helping me with this project and guiding me throughout and helping me in every step of the project.

I would also like to thank God Almighty and my parents for always being by my side and my fellow mates who were always ready to help.

TABLE OF CONTENTS

S.No	TITLE	PAGE No
1	INTRODUCTION	4
2	PROJECT SELECTION	5
3	WORKING ENVIRONMENT	6
4	LIBRARIES & MODULES	7
5	DATA DICTIONARY	8 - 10
6	SOURCE CODE	11 - 38
7	LOG OF PROJECT	39 - 41
8	SAMPLE OUTPUT	42 - 52
9	BIBLIOGRAPHY	53

INTRODUCTION

This project focuses on the development of a **Teacher Substitution and Timetable Management System**, designed to simplify and automate the process of managing school timetables, teacher attendance, and substitution allotment. The application provides a convenient digital platform that reduces manual paperwork and ensures smooth academic scheduling within the institution.

The system allows teachers to securely log in to their accounts and access features such as viewing their personalized timetable and applying for leave. Leave applications are automatically recorded and stored, and once approved by the coordinator, the system intelligently assigns substitute teachers by checking availability.

The coordinator (admin) is given full control over the monitoring and approval process. They can view all teachers' timetables, manage leave requests, and review the substitution log generated by the system. The program uses a combination of MySQL database storage and text/CSV file fallback options, making it flexible for environments with or without database support.

The inspiration for this project came from observing how schools traditionally handle teacher scheduling manually. By integrating digital automation—similar to modern school management systems—we aimed to create a reliable, user-friendly application that improves efficiency and reduces scheduling conflicts.

PROJECT SELECTION

We chose this project with the aim of creating an efficient and interactive platform that helps schools manage teacher timetables, leave applications, and substitution duties in a streamlined manner. The application provides a smooth user experience through a visually simple and intuitive interface built using Python's Tkinter library. To ensure reliable data handling, both Python file operations and MySQL database concepts have been incorporated into the system.

The administrator maintains a database consisting of multiple tables such as **Teachers**, **Subjects**, **Classes**, **Timetable**, and **Leaves**. Teachers can conveniently view their daily timetable, apply for leave, and track their schedules through an easily accessible dashboard. The coordinator's experience is further enhanced through features like automated substitute teacher assignment, leave approval, and viewing complete school timetables. All functionalities are made possible through seamless retrieval and updating of information in the database or fallback data files.

WORKING ENVIRONMENT

OPTIMUM REQUIREMENTS

- **Operating System** – Windows 10
- **Processor** – Must be clocked over 1.5 GHz
- **Graphics Driver** – Intel Integrated Graphics or equivalent or greater
- **RAM** – 8 GB or more.
- **Hard Disk** – 1 TB
- **Python interpreter** – Python IDLE 3.6
- **MySQL**

LIBRARIES & MODULES

Module/Library	Purpose
tkinter as tk	Standard Python library for creating the graphical user interface (GUI), forming the application's windows and core components.
font	Used to create and apply custom font styles (e.g., bold, italic) to text elements within the GUI for styling.
ttk	Provides the Themed Tkinter Widgets for modern-looking GUI components like styled buttons.
messagebox	Used to display standard dialog boxes for communication, such as errors, warnings, success messages, and notifications.
csv	Used for reading from and writing data to CSV and text files, specifically for managing user credentials, leave requests, and substitution logs.
os	Provides an interface for interacting with the operating system, primarily used to check if necessary files (like users.csv) exist.
random	Used to generate random numbers for non-critical tasks like drawing the space theme and for fair selection of substitute teachers.
datetime	Used for date and time manipulation, specifically to convert date strings (DD/MM/YYYY) into full day names for timetable lookup.
mysql.connector	The official driver used to connect to, query, and retrieve timetable data from the configured MySQL database.
Error	A specific class from the mysql.connector module used for catching and handling database connection and query exceptions.

Data Dictionary

USER DEFINED FUNCTIONS

Project.py	
Function Name	Purpose
draw_space	Draws the space-themed graphics (stars, planets) on the login canvas for visual effect.
ensure_user_file	Verifies the existence of users.csv and creates it with default admin and teacher credentials if missing.
validate_login	Reads user credentials from users.csv to authenticate the user and return their role.
save_leave	Appends a new leave request record to the leaves.txt file with a "Pending" status.
get_pending_leaves	Retrieves and returns a list of all leave requests that are currently marked as "Pending".
approve_leave	Changes a specific leave record's status to "Approved" in leaves.txt and calls the substitution logic.
get_day_name_from_date_str	Converts a date string in DD/MM/YYYY format into its corresponding full day name (e.g., "Tuesday").
read_substitution_log	Reads all substitution entries from the substitution_log.csv file.
log_substitution	Appends a new substitution assignment record to the substitution_log.csv file.
delete_substitution_record	Locates and removes a specific substitution entry from

	substitution_log.csv based on all four matching fields.
assign_substitute	Implements the core logic to find an available teacher to cover the absent teacher's periods on a specific day.
connect_mysql	Attempts to establish a connection to the configured MySQL database.
fetch_timetable_from_mysql	Queries the connected MySQL database to retrieve timetable data for a specified teacher or class.
fetch_timetable_from_csv	Reads all timetable data from the fallback timetable.csv file.
fetch_timetable_from_db	Determines the correct data source (MySQL or CSV) and fetches the relevant timetable information.
MainApp.__init__	Initializes the main graphical application window and sets up the primary layout components (header, sidebar, content).
MainApp.clear_content	Clears the main content area by destroying all its current widgets.
MainApp.clear_sidebar	Clears the sidebar area by destroying all its current widgets.
MainApp.show_login	Displays the application's login interface.
MainApp.attempt_login	Executes the login process, handles success/failure, sets user data, and directs the user to their dashboard.
MainApp.check_substitution_notifications	Displays an alert to the logged-in teacher if they have been assigned a substitution for today or a future date.
MainApp.build_sidebar	Populates the navigation sidebar with role-appropriate buttons (Admin or Teacher).

MainApp.go_to_home	Directs the user back to their respective main dashboard view.
MainApp.logout	Logs the current user out of the system and returns to the login screen.
MainApp.display_timetable_grid	Creates and displays the visual, color-coded, grid-based view of a timetable.
MainApp.show_teacher_overview	Displays the main welcome and navigation dashboard for a teacher.
MainApp.show_my_timetable	Fetches and displays the timetable for the currently logged-in teacher.
MainApp.show_apply_leave	Displays the graphical form for a teacher to submit a leave request.
MainApp.show_admin_overview	Displays the main dashboard for the Admin, summarizing key information like pending leaves.
MainApp.show_pending_leaves	Displays a list in a Treeview of all pending leave requests, allowing the Admin to approve them.
MainApp.show_all_teachers_timetables	Displays a menu that allows the Admin to select and view the timetable of any teacher.
MainApp.show_single_teacher_timetable	Fetches and displays the timetable for a specific teacher selected by the Admin.
MainApp.show_all_classes_timetables	Displays a menu that allows the Admin to select and view the timetable of any class.
MainApp.show_single_class_timetable	Fetches and displays the timetable for a specific class selected by the Admin.
MainApp.show_substitution_log	Displays the complete list of all substitution records, including the delete option for Admin users.

SOURCE CODE

```
import tkinter as tk
from tkinter import font, ttk, messagebox
import csv
import os
import random
from datetime import datetime
import mysql.connector
from mysql.connector import Error

SUBJECT_COLORS = {
    "Eng": "#A4C2F4",
    "EVS": "#B6D7A8",
    "Math": "#FFD966",
    "Hindi": "#D9D2E9",
    "Games": "#FCE5CD",
    "SST": "#F4CCCC",
    "Art": "#F7A8D8",
    "_default": "#CCCCCC"
}

CLASSES = ['1a', '1b', '1c']
SUBJECTS = ["Eng", "EVS", "Math", "Hindi", "Games", "SST", "Art"]

CLASS_TEACHERS = {
    "1a": "eng_teacher",
    "1b": "evs_teacher",
    "1c": "math_teacher"
}

TEACHER_TABLE_MAP = {
    "eng_teacher": "engteach",
    "evs_teacher": "eteach",
    "math_teacher": "mteach",
    "hindi_teacher": "hteach",
    "games_teacher": "gteach",
    "sst_teacher": "steach",
    "art_teacher": "ateach"
}
```

```

TEACHER_TO_SUBJECT_MAP = {
    "eng_teacher": "Eng",
    "evs_teacher": "EVS",
    "math_teacher": "Math",
    "hindi_teacher": "Hindi",
    "games_teacher": "Games",
    "sst_teacher": "SST",
    "art_teacher": "Art"
}

def draw_space(canvas, width, height):
    for _ in range(150):
        x = random.randint(0, width)
        y = random.randint(0, height)
        size = random.randint(1, 3)
        canvas.create_oval(x, y, x+size, y+size, fill="#00fffa", outline="")

planets = [
    ("#ff6f61", "#cc594f", 100, 100, 50, False),
    ("#6fa8dc", "#598fb0", 300, 200, 70, True),
    ("#ffd966", "#ccac52", 200, 350, 40, False),
]

for color, shadow_color, x, y, r, has_ring in planets:
    offset = r * 0.25
    shadow_r_offset = r * 0.05
    canvas.create_oval(
        x - r + offset, y - r + offset,
        x + r - shadow_r_offset, y + r - shadow_r_offset,
        fill=shadow_color, outline=shadow_color
    )
    canvas.create_oval(x-r, y-r, x+r, y+r, fill=color, outline=color)
    highlight_r = r * 0.15
    canvas.create_oval(
        x - r * 0.6, y - r * 0.6,
        x - r * 0.6 + highlight_r, y - r * 0.6 + highlight_r,
        fill="ffffff", outline=""
    )
    if has_ring:
        ring_width = r * 0.35

```

```

canvas.create_oval(
    x - r - ring_width, y - r * 0.3,
    x + r + ring_width, y + r * 0.3,
    outline="#ffffff", fill="", width=2
)

```

```

USERS_FILE = "users.csv"
TIMETABLE_FILE = "timetable.csv"
LEAVES_FILE = "leaves.txt"
SUB_LOG = "substitution_log.csv"

```

```

DB_CONFIG = {
    "host": "localhost",
    "user": "root",
    "password": "its_a3n0",
    "database": "slss",
    "auth_plugin": "mysql_native_password",
    "port": 3306,
    "raise_on_warnings": True,
    "autocommit": True
}

```

```

def ensure_user_file():
    if not os.path.exists(USERS_FILE):
        with open(USERS_FILE, "w", newline="") as f:
            writer = csv.writer(f)
            writer.writerow(["admin", "admin123", "admin"])
            writer.writerow(["eng_teacher", "engpass", "teacher"])
            writer.writerow(["evs_teacher", "evspass", "teacher"])
            writer.writerow(["math_teacher", "mathpass", "teacher"])
            writer.writerow(["hindi_teacher", "hindipass", "teacher"])
            writer.writerow(["games_teacher", "gamespass", "teacher"])
            writer.writerow(["sst_teacher", "sstpass", "teacher"])
            writer.writerow(["art_teacher", "artpass", "teacher"])

```

```

def validate_login(username, password):
    try:
        with open(USERS_FILE, "r") as f:
            for row in csv.reader(f):
                if len(row) >= 3 and row[0].lower() == username.lower() and row[1] == password:
                    return row[2]

```

```

except FileNotFoundError:
    return None
return None

def save_leave(teacher, date_str, reason):
    with open(LEAVES_FILE, "a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow([teacher, date_str, reason, "Pending"])

def get_pending_leaves():
    leaves = []
    if os.path.exists(LEAVES_FILE):
        with open(LEAVES_FILE, "r", newline="") as f:
            for row in csv.reader(f):
                if len(row) >= 4 and row[3] == "Pending":
                    leaves.append(row)
    return leaves

def approve_leave(teacher, date_str):
    updated = []
    found = False
    if os.path.exists(LEAVES_FILE):
        with open(LEAVES_FILE, "r", newline="") as f:
            rows = list(csv.reader(f))
            for row in rows:
                if len(row) >= 4 and row[0].lower() == teacher.lower() and row[1] == date_str and row[3]
== "Pending":
                    row[3] = "Approved"
                    found = True
                    updated.append(row)
        with open(LEAVES_FILE, "w", newline="") as f:
            csv.writer(f).writerows(updated)
    if found:
        assign_substitute(teacher, date_str)
    return found

def get_day_name_from_date_str(date_str):
    try:
        dt = datetime.strptime(date_str, "%d/%m/%Y")
        return dt.strftime('%A')
    except ValueError:

```

```

return date_str

def assign_substitute(absent_teacher, date_str):
    absent_teacher_lower = absent_teacher.lower()
    required_day_name = get_day_name_from_date_str(date_str)

    all_teachers = set()
    if os.path.exists(USERS_FILE):
        with open(USERS_FILE, "r") as f:
            for row in csv.reader(f):
                if len(row) >= 3 and row[2] == "teacher" and row[0].lower() in
TEACHER_TABLE_MAP:
                    all_teachers.add(row[0])

    available_candidates = [t for t in all_teachers if t.lower() != absent_teacher_lower]
    absent_tt = fetch_timetable_from_db(teacher_name=absent_teacher)
    periods_to_substitute = []

    for day, period_no, absent_subj, classn in absent_tt:
        if day.lower() == required_day_name.lower():
            if classn.strip() and classn.lower() not in ("free", "none", "-", "null"):
                periods_to_substitute.append((day, period_no, classn, absent_subj))

    if not periods_to_substitute:
        messagebox.showwarning("Substitution Error", f"No periods requiring substitution for
{absent_teacher} on {required_day_name}.")
        return []

    busy_teachers = {}

    for candidate in available_candidates:
        candidate_tt = fetch_timetable_from_db(teacher_name=candidate)
        for day, period_no, subject, classn in candidate_tt:
            if day.lower() == required_day_name.lower():
                if not classn.strip() or classn.lower() in ("free", "none", "-", "null"):
                    continue
                key = (day.lower(), period_no)
                if key not in busy_teachers:
                    busy_teachers[key] = set()
                busy_teachers[key].add(candidate.lower())

```

```

assigned = []
existing_subs = read_substitution_log()

try:
    with open(SUB_LOG, "a", newline="") as f:
        writer = csv.writer(f)
        substitutes_used_today = set()

        for day, period_no, classn, absent_subj in periods_to_substitute:
            if [r for r in existing_subs if len(r) > 2 and r[0] == date_str and r[1] == period_no and
                r[2].lower() == absent_teacher_lower]:
                continue

            key = (day.lower(), period_no)
            free_teachers = [
                t for t in available_candidates
                if t.lower() not in busy_teachers.get(key, set())
            ]

            sub = "No Substitute Available"
            primary_candidates = [
                t for t in free_teachers
                if t.lower() != 'art_teacher' and t.lower() not in substitutes_used_today
            ]

            if primary_candidates:
                sub = random.choice(primary_candidates)
                substitutes_used_today.add(sub.lower())
            elif 'art_teacher' in [t.lower() for t in free_teachers]:
                sub = next((t for t in free_teachers if t.lower() == 'art_teacher'), 'art_teacher')
            elif free_teachers:
                sub = random.choice(free_teachers)

            if sub != "No Substitute Available":
                writer.writerow([date_str, period_no, absent_teacher, sub])
                assigned.append((date_str, period_no, absent_teacher, sub))

except PermissionError:
    messagebox.showerror("File Error", f"Permission denied to write to {SUB_LOG}.")
    return []

```

return assigned

```
def read_substitution_log():
    rows = []
    if not os.path.exists(SUB_LOG):
        try:
            with open(SUB_LOG, "w", newline="") as f:
                writer = csv.writer(f)
                writer.writerow(["Date", "Period", "Absent", "Substitute"])
        except PermissionError:
            messagebox.showerror("File Error", f"Permission denied to create {SUB_LOG}.")
            return []
        return []

    try:
        with open(SUB_LOG, "r", newline="") as f:
            reader = csv.reader(f)
            for row in reader:
                if not row:
                    continue
                if len(row) >= 1 and row[0].strip().lower() == "date":
                    continue
                normalized = [col.strip() for col in row]
                if len(normalized) >= 4:
                    rows.append(normalized[:4])
    except PermissionError:
        messagebox.showerror("File Error", f"Permission denied to read {SUB_LOG}.")
        return []
    except Exception:
        return []

    return rows

def delete_substitution_record(date_str, period, absent, sub):
    try:
        rows = read_substitution_log()
        if not rows:
            return False

        target_date = date_str.strip().lower()
        target_period = str(period).strip()
```

```

target_absent = absent.strip().lower()
target_sub = sub.strip().lower()

updated = []
deleted_any = False

for r in rows:
    r_date = r[0].strip().lower()
    r_period = str(r[1]).strip()
    r_absent = r[2].strip().lower()
    r_sub = r[3].strip().lower()

    if r_date == target_date and r_period == target_period and r_absent == target_absent and
r_sub == target_sub:
        deleted_any = True
        continue
    updated.append([r[0].strip(), r[1].strip(), r[2].strip(), r[3].strip()])

with open(SUB_LOG, "w", newline="") as f:
    writer = csv.writer(f)
    writer.writerow(["Date", "Period", "Absent", "Substitute"])
    writer.writerows(updated)

return deleted_any
except PermissionError:
    messagebox.showerror("File Error", f"Permission denied to write to {SUB_LOG}.")
    return False
except Exception:
    return False

def connect_mysql():
    try:
        conn = mysql.connector.connect(**DB_CONFIG)
        if conn.is_connected():
            return conn
    except Error:
        return None
    return None

def list_tables(conn):
    try:

```

```

cur = conn.cursor()
cur.execute("SHOW TABLES;")
rows = cur.fetchall()
return [r[0] for r in rows]
except:
    return []

def fetch_timetable_from_mysql(conn, table_names=None, is_teacher_tt=False,
teacher_name=None):
    rows = []
    if not table_names:
        return rows

    try:
        cur = conn.cursor()
        for tbl in table_names:
            cur.execute(f"SHOW COLUMNS FROM `{tbl}`;")
            cols_info = cur.fetchall()
            cols = [c[0].lower() for c in cols_info]

            day_col = next((c for c in cols if c == 'day'), None)
            if not day_col:
                continue

            period_cols = [c for c in cols if c.startswith('period')]
            if not period_cols:
                continue

            cur.execute(f"SELECT * FROM `{tbl}`;")
            records = cur.fetchall()
            orig_cols = [c[0] for c in cols_info]

            for record in records:
                rec_map = {}
                for i, c in enumerate(orig_cols):
                    rec_map[c.lower()] = record[i] if i < len(record) else None

                day_val = str(rec_map.get(day_col, "")).strip()

                for pcol in period_cols:
                    entry = str(rec_map.get(pcol, "")).strip() if rec_map.get(pcol, "") is not None else ""

```

```

if not entry or entry.upper() in ('BREAK','LUNCH'):
    continue

pnum = ".join([ch for ch in pcol if ch.isdigit()])
try:
    pnum_int = int(pnum)
    if not (1 <= pnum_int <= 9):
        continue
    pnum = str(pnum_int)
except ValueError:
    continue

if is_teacher_tt:
    class_name = entry
    subject = TEACHER_TO_SUBJECT_MAP.get(teacher_name.lower(), 'Unknown
Subject')
    class_name = class_name if class_name else ""
else:
    class_name = tbl
    subject = entry
    if not subject: continue

    rows.append((day_val, pnum, subject, class_name))
except:
    return rows
return rows

def fetch_timetable_from_csv():
    data = []
    if os.path.exists(TIMETABLE_FILE):
        with open(TIMETABLE_FILE, "r", newline="") as f:
            reader = csv.reader(f)
            for row in reader:
                if len(row) < 4: continue
                date, period, subject, class_name = [c.strip() for c in row[:4]]
                data.append((date, period, subject, class_name))
    else:
        days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]
        for class_name in CLASSES:
            for day in days:
                for period in range(1, 10):

```

```

        subj = random.choice(SUBJECTS)
        data.append((day, str(period), subj, class_name))
    return data

def fetch_timetable_from_db(teacher_name=None, class_name=None):
    conn = connect_mysql()

    if conn:
        try:
            if teacher_name:
                teacher_lower = teacher_name.lower()
                teacher_table = TEACHER_TABLE_MAP.get(teacher_lower)

                if teacher_table:
                    rows = fetch_timetable_from_mysql(conn, table_names=[teacher_table],
is_teacher_tt=True, teacher_name=teacher_name)
                    return rows
                else:
                    return []

            elif class_name:
                class_table = class_name.lower()
                rows = fetch_timetable_from_mysql(conn, table_names=[class_table],
is_teacher_tt=False)
                return rows

            else:
                class_tables = CLASSES
                rows = fetch_timetable_from_mysql(conn, table_names=class_tables,
is_teacher_tt=False)
                return rows

        except:
            return fetch_timetable_from_csv()
        finally:
            if conn and conn.is_connected():
                conn.close()

    else:
        if not os.path.exists(TIMETABLE_FILE):
            pass

```

```

    return fetch_timetable_from_csv()

def is_teacher_free(teacher_name, date_str, period):
    day = get_day_name_from_date_str(date_str)
    tt = fetch_timetable_from_db(teacher_name=teacher_name)

    for d, p, subj, classn in tt:
        if d.lower() == day.lower() and str(p) == str(period):
            if not classn.strip() or classn.lower() in ("free", "none", "-", "null"):
                return True
            else:
                return False

    return True

def update_substitution(date_str, period, new_teacher):
    rows = read_substitution_log()
    updated = False

    for row in rows:
        if len(row) >= 4:
            if row[0] == date_str and row[1] == str(period):
                row[3] = new_teacher
                updated = True

    if updated:
        with open(SUB_LOG, "w", newline="") as f:
            writer = csv.writer(f)
            writer.writerow(["Date", "Period", "Absent", "Substitute"])
            writer.writerows(rows)
        return True
    return False

class MainApp(tk.Tk):
    def __init__(self):
        super().__init__()
        self.title("Teacher Substitution System")
        self.geometry("1000x600")
        self.minsize(900, 520)
        self.configure(bg="#11122a")

```

```

self.style = ttk.Style(self)
try:
    self.style.theme_use("clam")
except:
    pass
self.style.configure("TFrame", background="#11122a")
self.style.configure("Header.TLabel", font=("Segoe UI", 16, "bold"),
background="#0a0a23", foreground="#00ffa")
self.style.configure("Sidebar.TButton", font=("Segoe UI", 11), padding=8)
self.style.configure("Card.TLabel", background="#1b1c33", font=("Segoe UI", 10))
self.style.map("Sidebar.TButton",
               background=[("active", "#222244"), ("!active", "#1b1c33")],
               foreground=[("active", "#00ffa"), ("!active", "#00ffa")])

self.current_user = None
self.current_role = None

self.header = tk.Frame(self, bg="#0a0a23", height=70)
self.header.pack(side="top", fill="x")
self.header.pack_propagate(False)
self.logo = tk.Label(self.header, text="School Substitution System", fg="#00ffa",
                     bg="#0a0a23", font=("Segoe UI", 18, "bold"))
self.logo.pack(side="left", padx=20)
self.user_label = tk.Label(self.header, text="", fg="#00ffa", bg="#0a0a23", font=("Segoe
UI", 11))
self.user_label.pack(side="right", padx=20)

body = tk.Frame(self, bg="#11122a")
body.pack(fill="both", expand=True)

self.sidebar = tk.Frame(body, width=240, bg="#1b1c33", relief="flat")
self.sidebar.pack(side="left", fill="y", padx=12, pady=12)
self.sidebar.pack_propagate(False)

self.content = tk.Frame(body, bg="#11122a")
self.content.pack(side="right", fill="both", expand=True, padx=12, pady=12)
self.content.pack_propagate(False)

self.show_welcome_screen()

def clear_sidebar(self):

```

```

for w in self.sidebar.winfo_children():
    w.destroy()

def clear_content(self):
    for w in self.content.winfo_children():
        w.destroy()

def show_welcome_screen(self):
    self.clear_sidebar()
    self.clear_content()

    canvas = tk.Canvas(self.content, bg="#0a0a23", highlightthickness=0)
    canvas.pack(fill="both", expand=True)

    draw_space(canvas, 1000, 600)

    def on_resize(event):
        canvas.delete("all")
        draw_space(canvas, event.width, event.height)

    canvas.bind("<Configure>", on_resize)

    overlay_frm = tk.Frame(canvas, bg="#0a0a23", bd=0)
    overlay_frm.place(relx=0.75, rely=0.5, anchor="center")

    title_text = "COSMOS\nACADEMY"
    tk.Label(overlay_frm, text=title_text,
             font=("Ink Free", 36, "bold", "italic"),
             bg="#0a0a23",
             fg="#00fffa",
             justify="left").pack(anchor="w", pady=(0, 5))

    tk.Label(overlay_frm, text="Explore the Universe of Teachers",
             font=("Segoe UI", 14, "italic"),
             bg="#0a0a23",
             fg="#00fffa").pack(anchor="w", pady=(0, 20))

    ttk.Button(overlay_frm, text="Explore Now",
               command=self.show_login,
               style="Sidebar.TButton",
               width=15).pack(anchor="w")

```

```

def show_login(self):
    self.clear_sidebar()
    self.clear_content()
    frm = tk.Frame(self.content, bg="#1b1c33", bd=0, relief="ridge")
    frm.place(relx=0.5, rely=0.45, anchor="center", width=480, height=300)

    lbl = tk.Label(frm, text="Sign in", font=("Segoe UI", 18, "bold"), bg="#1b1c33",
fg="#00fffa")
    lbl.pack(pady=(18, 10))

    tk.Label(frm, text="Username", bg="#1b1c33", fg="#00fffa", anchor="w").pack(fill="x",
padx=24)
    self.login_user = ttk.Entry(frm)
    self.login_user.pack(fill="x", padx=24, pady=6)

    tk.Label(frm, text="Password", bg="#1b1c33", fg="#00fffa", anchor="w").pack(fill="x",
padx=24)
    self.login_pwd = ttk.Entry(frm, show="*")
    self.login_pwd.pack(fill="x", padx=24, pady=6)

    btn = ttk.Button(frm, text="Login", command=self.perform_login, style="Sidebar.TButton")
    btn.pack(pady=12)

    hint = tk.Label(frm, text="Demo: admin/admin123; eng_teacher/engpass;
evs_teacher/evspass; math_teacher/mathpass; art_teacher/artpass", bg="#1b1c33", fg="#666")
    hint.pack(side="bottom", pady=8)

def perform_login(self):
    u = self.login_user.get().strip()
    p = self.login_pwd.get().strip()
    role = validate_login(u, p)
    if not role:
        messagebox.showerror("Login failed", "Invalid username or password")
        return
    self.current_user = u
    self.current_role = role
    self.user_label.config(text=f"{u} ({role})")
    self.build_sidebar()
    if role == "admin":
        self.show_admin_overview()

```

```

else:
    self.show_teacher_overview()

def build_sidebar(self):
    self.clear_sidebar()
    user_frame = tk.Frame(self.sidebar, bg="#1b1c33")
    user_frame.pack(fill="x", pady=(10, 15))
    avatar = tk.Label(user_frame, text=self.current_user[:1].upper(), bg="#222244",
fg="#00fffa",
                        font=("Segoe UI", 14, "bold"), width=2, height=1)
    avatar.pack(side="left", padx=12, pady=6)
    tk.Label(user_frame, text=self.current_user, bg="#1b1c33", fg="#00fffa", font=("Segoe UI",
11)).pack(side="left", padx=6)

    ttk.Separator(self.sidebar, orient="horizontal").pack(fill="x", pady=(6, 12))

    if self.current_role == "admin":
        buttons = [
            ("Dashboard", self.show_admin_overview),
            ("All Timetables (Teachers)", self.show_all_teachers_timetables),
            ("All Timetables (Classes)", self.show_all_classes_timetables),
            ("Pending Leaves", self.show_pending_leaves),
            ("Substitution Log", self.show_substitution_log),
            ("Logout", self.logout)
        ]
    else:
        buttons = [
            ("Dashboard", self.show_teacher_overview),
            ("My Timetable", self.show_my_timetable),
            ("Apply Leave", self.show_apply_leave),
            ("Substitution Log", self.show_substitution_log),
            ("Logout", self.logout)
        ]

    for (text, cmd) in buttons:
        b = ttk.Button(self.sidebar, text=text, command=cmd, style="Sidebar.TButton")
        b.pack(fill="x", padx=12, pady=6)

def show_teacher_overview(self):
    self.clear_content()
    tk.Label(self.content, text=f"Welcome, {self.current_user}", font=("Segoe UI", 16, "bold"),

```

```

        bg="#11122a", fg="#00fffa").pack(anchor="nw")
    tk.Label(self.content, text="Quick actions", bg="#11122a",
fg="#66fffa").pack(anchor="nw", pady=(6, 12))

    cards = tk.Frame(self.content, bg="#11122a")
    cards.pack(fill="both", expand=False)

    def small_card(title, subtitle, command):
        frm = tk.Frame(cards, bg="#1b1c33", bd=1, relief="flat", padx=14, pady=12)
        frm.pack(side="left", padx=12)
        tk.Label(frm, text=title, font=("Segoe UI", 12, "bold"), bg="#1b1c33",
fg="#00fffa").pack(anchor="w")
        tk.Label(frm, text=subtitle, font=("Segoe UI", 10), bg="#1b1c33",
fg="#66fffa").pack(anchor="w", pady=(6, 0))
        ttk.Button(frm, text="Open", command=command).pack(anchor="e", pady=(8,0))

    small_card("View Timetable", "See your classes", self.show_my_timetable)
    small_card("Apply Leave", "Request a leave", self.show_apply_leave)
    small_card("Substitution Log", "See substitutions", self.show_substitution_log)

    def display_timetable_grid(self, title_name, timetable_rows, class_table_view=False):
        self.clear_content()

        title = tk.Label(
            self.content,
            text=f"TIMETABLE – {title_name.upper()}",
            font=("Segoe UI", 16, "bold"),
            bg="#11122a",
            fg="#00fffa"
        )
        title.pack(anchor="nw", pady=(5, 10))

        days = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday"]

        GRID_SLOTS = ["P1", "P2", "P3", "BREAK", "P4", "P5", "P6", "LUNCH", "P7", "P8",
"P9"]

        PERIOD_TO_SLOT_INDEX = {
            1: 0, 2: 1, 3: 2, 4: 4, 5: 5,
            6: 6, 7: 8, 8: 9, 9: 10
        }

```

```

frame = tk.Frame(self.content, bg="#11122a")
frame.pack(pady=10)

tk.Label(frame, text="DAY", font=("Segoe UI", 11, "bold"),
         bg="#1b1c33", fg="#00fffa", width=10, relief="ridge").grid(row=0, column=0)

for i, slot in enumerate(GRID_SLOTS):
    tk.Label(
        frame, text=slot, font=("Segoe UI", 11, "bold"),
        bg="#1b1c33", fg="#00fffa", width=7, relief="ridge"
    ).grid(row=0, column=i+1)

grid_map = {day: {} for day in days}

for day, period_no, subject, class_name in timetable_rows:
    try:
        pnum_int = int(period_no)
        slot_index = PERIOD_TO_SLOT_INDEX.get(pnum_int)
        if slot_index is not None:
            if class_table_view:
                text_to_display = subject
            else:
                if not class_name or class_name.upper() in ('FREE', 'NONE', '-', 'NULL'):
                    text_to_display = "FREE"
                    color_key = "_default"
                else:
                    text_to_display = class_name
                    color_key = subject

            if text_to_display != "FREE":
                color_key = subject.strip()
                color = SUBJECT_COLORS.get(color_key, SUBJECT_COLORS["_default"])
            else:
                color = "#333333"

            grid_map[day][slot_index] = (text_to_display, color)
        except ValueError:
            continue

for r, day in enumerate(days, start=1):

```

```
tk.Label(frame, text=day, font=("Segoe UI", 10, "bold"),
        bg="#222244", fg="#FFFFFF", width=10, relief="raised").grid(row=r, column=0,
sticky="nsew")
```

```
for c, slot in enumerate(GRID_SLOTS, start=1):
    content = grid_map[day].get(c-1, (slot, "#2c2c2c"))
    text, bg_color = content
    fg_color = "#000000"
```

```
if text in ("BREAK", "LUNCH"):
    bg_color = "#444444"
    fg_color = "#FFFFFF"
elif text == "FREE":
    bg_color = "#333333"
    fg_color = "#AAAAAA"
elif bg_color == "#333333":
    fg_color = "#FFFFFF"
```

```
tk.Label(
    frame, text=text, font=("Segoe UI", 9, "bold"),
    bg=bg_color, fg=fg_color, width=7, height=2, wraplength=50,
    relief="groove"
).grid(row=r, column=c, sticky="nsew", padx=1, pady=1)
```

```
def show_my_timetable(self):
    tt_rows = fetch_timetable_from_db(teacher_name=self.current_user)
    self.display_timetable_grid(f'My
({TEACHER_TO_SUBJECT_MAP.get(self.current_user.lower(),
                                ")}"),
class_table_view=False)
```

Timetable
tt_rows,

```
def show_apply_leave(self):
    self.clear_content()
    tk.Label(self.content, text="Apply for Leave", font=("Segoe UI", 16, "bold"),
            bg="#11122a", fg="#00fffa").pack(anchor="nw", pady=(5, 10))
```

```
frm = tk.Frame(self.content, bg="#1b1c33", padx=20, pady=20)
frm.pack(pady=10, fill="x")
```

```
tk.Label(frm, text="Date (DD/MM/YYYY):", bg="#1b1c33", fg="#00fffa").grid(row=0,
column=0, sticky="w", pady=5)
self.leave_date = ttk.Entry(frm, width=30)
```

```

self.leave_date.grid(row=0, column=1, sticky="w", padx=10, pady=5)
self.leave_date.insert(0, datetime.now().strftime("%d/%m/%Y"))

tk.Label(frm, text="Reason:", bg="#1b1c33", fg="#00fffa").grid(row=1, column=0,
sticky="w", pady=5)
self.leave_reason = tk.Text(frm, width=30, height=5)
self.leave_reason.grid(row=1, column=1, sticky="w", padx=10, pady=5)

ttk.Button(frm, text="Submit Leave", command=self.submit_leave).grid(row=2, column=1,
sticky="e", pady=10)

def submit_leave(self):
    date_str = self.leave_date.get().strip()
    reason = self.leave_reason.get("1.0", tk.END).strip()

    try:
        dt = datetime.strptime(date_str, "%d/%m/%Y")
        if dt < datetime.now().replace(hour=0, minute=0, second=0, microsecond=0):
            messagebox.showerror("Error", "Cannot apply for a past date.")
            return
    except ValueError:
        messagebox.showerror("Error", "Invalid date format.")
        return

    if not reason:
        messagebox.showerror("Error", "Please provide a reason.")
        return

    save_leave(self.current_user, date_str, reason)
    messagebox.showinfo("Success", f"Leave submitted for {date_str}.")
    self.show_teacher_overview()

def show_admin_overview(self):
    self.clear_content()
    tk.Label(self.content, text=f"Admin Dashboard", font=("Segoe UI", 16, "bold"),
            bg="#11122a", fg="#00fffa").pack(anchor="nw")

    pending_leaves = get_pending_leaves()

    tk.Label(self.content, text=f"Pending Leaves: {len(pending_leaves)}",

```

```

font=("Segoe UI", 14), bg="#11122a", fg="#ff6f61").pack(anchor="nw", pady=(10,
5))

ttk.Button(self.content, text="Review Leaves Now",
command=self.show_pending_leaves).pack(anchor="nw", pady=(0, 20))

tk.Label(self.content, text="System Timetables", font=("Segoe UI", 14),
bg="#11122a", fg="#00fffa").pack(anchor="nw", pady=(10, 5))

cards = tk.Frame(self.content, bg="#11122a")
cards.pack(fill="x", expand=False)

def admin_card(title, subtitle, command):
    frm = tk.Frame(cards, bg="#1b1c33", bd=1, relief="flat", padx=14, pady=12)
    frm.pack(side="left", padx=12, fill="x", expand=True)
    tk.Label(frm, text=title, font=("Segoe UI", 12, "bold"), bg="#1b1c33",
fg="#00fffa").pack(anchor="w")
    tk.Label(frm, text=subtitle, font=("Segoe UI", 10), bg="#1b1c33",
fg="#66fffa").pack(anchor="w", pady=(6, 0))
    ttk.Button(frm, text="View", command=command).pack(anchor="e", pady=(8,0))

    admin_card("Teacher Timetables", "View individual teacher schedules",
self.show_all_teachers_timetables)
    admin_card("Class Timetables", "View class-wise schedules",
self.show_all_classes_timetables)
    admin_card("Substitution Log", "Review all past substitutions", self.show_substitution_log)

def show_all_teachers_timetables(self):
    self.clear_content()
    tk.Label(self.content, text="All Teacher Timetables", font=("Segoe UI", 16, "bold"),
bg="#11122a", fg="#00fffa").pack(anchor="nw", pady=(5, 10))

    teacher_names = sorted(TEACHER_TABLE_MAP.keys())

    teacher_frame = tk.Frame(self.content, bg="#11122a")
    teacher_frame.pack(fill="x")

    for i, t_name in enumerate(teacher_names):
        display_name = t_name.replace('_', ' ').title()
        frm = tk.Frame(teacher_frame, bg="#1b1c33", bd=1, relief="flat", padx=10, pady=8)
        frm.grid(row=i//3, column=i%3, padx=10, pady=10, sticky="nsew")

```

```
tk.Label(frm, text=display_name, font=("Segoe UI", 12, "bold"), bg="#1b1c33",
fg="#00fffa").pack(anchor="w")
```

```
cmd = lambda name=t_name: self.show_single_teacher_timetable(name)
ttk.Button(frm, text="View TT", command=cmd).pack(anchor="e", pady=(5,0))
```

```
teacher_frame.grid_columnconfigure(0, weight=1)
teacher_frame.grid_columnconfigure(1, weight=1)
teacher_frame.grid_columnconfigure(2, weight=1)
```

```
def show_single_teacher_timetable(self, teacher_name):
    tt_rows = fetch_timetable_from_db(teacher_name=teacher_name)
    display_name = teacher_name.replace('_', ' ').title()
    self.display_timetable_grid(f'{display_name} Timetable', tt_rows, class_table_view=False)
```

```
def show_all_classes_timetables(self):
    self.clear_content()
    tk.Label(self.content, text="All Class Timetables", font=("Segoe UI", 16, "bold"),
            bg="#11122a", fg="#00fffa").pack(anchor="nw", pady=(5, 10))
```

```
class_frame = tk.Frame(self.content, bg="#11122a")
class_frame.pack(fill="x")
```

```
for i, class_name in enumerate(CLASSES):
    frm = tk.Frame(class_frame, bg="#1b1c33", bd=1, relief="flat", padx=10, pady=8)
    frm.grid(row=0, column=i, padx=10, pady=10, sticky="nsew")
```

```
tk.Label(frm, text=f'Class {class_name.upper()}', font=("Segoe UI", 12, "bold"),
bg="#1b1c33", fg="#00fffa").pack(anchor="w")
```

```
cmd = lambda name=class_name: self.show_single_class_timetable(name)
ttk.Button(frm, text="View TT", command=cmd).pack(anchor="e", pady=(5,0))
```

```
class_frame.grid_columnconfigure(0, weight=1)
class_frame.grid_columnconfigure(1, weight=1)
class_frame.grid_columnconfigure(2, weight=1)
```

```
def show_single_class_timetable(self, class_name):
    tt_rows = fetch_timetable_from_db(class_name=class_name)
```

```

        self.display_timetable_grid(f'Class      {class_name.upper()}      Timetable",      tt_rows,
class_table_view=True)

def show_pending_leaves(self):
    self.clear_content()
    tk.Label(self.content, text="Pending Leave Requests", font=("Segoe UI", 16, "bold"),
        bg="#111122a", fg="#00fffa").pack(anchor="nw", pady=(5, 10))

    leaves = get_pending_leaves()

    if not leaves:
        tk.Label(self.content, text="No pending leave requests.", bg="#111122a",
fg="#FFFFFF").pack(anchor="nw", pady=20)
        return

    tree_frame = tk.Frame(self.content, bg="#111122a")
    tree_frame.pack(fill="both", expand=True, pady=10)

    tree = ttk.Treeview(tree_frame, columns=("Teacher", "Date", "Reason", "Action"),
show="headings", height=10)
    tree.heading("Teacher", text="Teacher")
    tree.heading("Date", text="Date")
    tree.heading("Reason", text="Reason")
    tree.heading("Action", text="Action")

    tree.column("Teacher", width=150)
    tree.column("Date", width=100)
    tree.column("Reason", width=350)
    tree.column("Action", width=100)

    for i, (teacher, date_str, reason, status) in enumerate(leaves):
        tree.insert("", tk.END, values=(teacher.title(), date_str, reason, "Approve"), tags=(teacher,
date_str))

    tree.pack(fill="both", expand=True)

def on_tree_select(event):
    selected_item = tree.focus()
    if not selected_item:
        return

```

```

teacher, date_str = tree.item(selected_item, 'tags')

if messagebox.askyesno("Confirm Approval", f"Approve leave for {teacher} on
{date_str}?"):
    if approve_leave(teacher, date_str):
        messagebox.showinfo("Success", f"Leave approved.")
        self.show_pending_leaves()
    else:
        messagebox.showerror("Error", "Could not approve leave.")

tree.bind('<Double-1>', on_tree_select)

tk.Label(self.content, text="Double-click a row to approve leave.",
        bg="#111122a", fg="#66fffa").pack(anchor="nw", pady=10)

def show_substitution_log(self):
    self.clear_content()
    tk.Label(self.content, text="Substitution Log", font=("Segoe UI", 16, "bold"),
            bg="#111122a", fg="#00fffa").pack(anchor="nw", pady=(5, 10))

    log_entries = read_substitution_log()

    if not log_entries:
        tk.Label(self.content, text="No substitution records found.", bg="#111122a",
fg="#FFFFFF").pack(anchor="nw", pady=20)
        return

    if self.current_role != 'admin':
        log_entries = [
            row for row in log_entries
            if row[2].lower() == self.current_user.lower() or row[3].lower() ==
self.current_user.lower()
        ]
        if not log_entries:
            tk.Label(self.content, text="No substitution records found for you.", bg="#111122a",
fg="#FFFFFF").pack(anchor="nw", pady=20)
            return

    tree_frame = tk.Frame(self.content, bg="#111122a")
    tree_frame.pack(fill="both", expand=True, pady=10)

```

```

columns = ("Date", "Period", "Absent Teacher", "Substitute Teacher")
if self.current_role == "admin":
    columns += ("Delete",)

tree = ttk.Treeview(tree_frame, columns=columns, show="headings", height=15)
for col in columns:
    tree.heading(col, text=col)

if self.current_role == "admin":
    update_frame = tk.Frame(self.content, bg="#1b1c33", padx=10, pady=10)
    update_frame.pack(fill="x", pady=(5, 5))

    tk.Label(update_frame, text="Update Substitution:", bg="#1b1c33", fg="#00fffa",
             font=("Segoe UI", 12, "bold")).grid(row=0, column=0, columnspan=4, pady=5)

    tk.Label(update_frame, text="Date      (DD/MM/YYYY):", bg="#1b1c33",
             fg="#00fffa").grid(row=1, column=0)
    date_entry = ttk.Entry(update_frame, width=15)
    date_entry.grid(row=1, column=1, padx=5)

    tk.Label(update_frame, text="Period:", bg="#1b1c33", fg="#00fffa").grid(row=1,
    column=2)
    period_entry = ttk.Entry(update_frame, width=10)
    period_entry.grid(row=1, column=3, padx=5)

    tk.Label(update_frame, text="New Teacher Username:", bg="#1b1c33",
             fg="#00fffa").grid(row=2, column=0)
    teacher_entry = ttk.Entry(update_frame, width=20)
    teacher_entry.grid(row=2, column=1, padx=5)

    def perform_update():
        if self.current_role != "admin":
            messagebox.showerror("Access Denied", "Only admin can update.")
            return

        date_str = date_entry.get().strip()
        period = period_entry.get().strip()
        new_teacher = teacher_entry.get().strip().lower()

        if not date_str or not period or not new_teacher:
            messagebox.showerror("Error", "Please fill all fields.")

```

```

    return

all_teachers = list(TEACHER_TABLE_MAP.keys())
if new_teacher not in all_teachers:
    messagebox.showerror("Error", "Teacher does not exist.")
    return

if not is_teacher_free(new_teacher, date_str, period):
    messagebox.showerror("Busy Teacher", f"{new_teacher} already has a class.")
    return

if update_substitution(date_str, period, new_teacher):
    messagebox.showinfo("Success", "Substitution updated.")
    self.show_substitution_log()
else:
    messagebox.showerror("Error", "Record not found.")

    ttk.Button(update_frame, text="Update", command=perform_update).grid(row=2,
column=3, padx=5)

if self.current_role == "admin":

    def on_tree_click(event):
        item = tree.focus()
        if not item:
            return

        values = tree.item(item)["values"]
        if not values:
            return

        if len(values) == 5 and values[4] == "Delete":
            date_disp = str(values[0]).strip()
            period_disp = str(values[1]).strip()
            absent_disp = str(values[2]).strip()
            sub_disp = str(values[3]).strip()

            if not messagebox.askyesno("Confirm Delete",
                                     f"Delete this record?\n\nDate: {date_disp}\nPeriod:
{period_disp}\nAbsent: {absent_disp}\nSub: {sub_disp}"):
                return

```

```

    ok = delete_substitution_record(date_disp, period_disp, absent_disp, sub_disp)
    if ok:
        messagebox.showinfo("Deleted", "Record removed.")
    else:
        messagebox.showerror("Not found", "Could not find record.")
    self.show_substitution_log()

tree.bind("<Double-1>", on_tree_click)

tree.heading("Date", text="Date")
tree.heading("Period", text="Period")
tree.heading("Absent Teacher", text="Absent Teacher")
tree.heading("Substitute Teacher", text="Substitute Teacher")

tree.column("Date", width=120)
tree.column("Period", width=80)
tree.column("Absent Teacher", width=180)
tree.column("Substitute Teacher", width=180)
if self.current_role == "admin":
    tree.column("Delete", width=100)

for date_str, period, absent, sub in log_entries:
    display_row = (date_str, period, absent.title(), sub.title())
    if self.current_role == "admin":
        display_row += ("Delete",)

    tags = ()
    if self.current_role != 'admin' and sub.lower() == self.current_user.lower():
        tags = ('highlight',)

    tree.insert("", tk.END, values=display_row, tags=tags)

tree.tag_configure('highlight', background='#00fffa', foreground='black')

tree.pack(fill="both", expand=True)

def logout(self):
    if messagebox.askyesno("Logout", "Are you sure?"):
        self.current_user = None
        self.current_role = None

```

```
self.user_label.config(text="")  
self.show_login()  
  
if __name__ == "__main__":  
    ensure_user_file()  
    app = MainApp()  
    app.mainloop()
```

LOG OF PROJECT

LOG-1: 16/06/2025

- Ideation
- Discussion of topic – Teacher Substitution & Timetable Management System

LOG-2: 23/06/2025

- Searched for suitable GUI modules (Tkinter)
- Explored data storage options (CSV, Text Files, MySQL)

LOG-3: 30/06/2025

- Decision to use MySQL for structured storage
- Studied Python–MySQL connectivity

LOG-4: 07/07/2025

- Created SQL tables: Teachers, Subjects, Classes
- Worked on Login Window GUI
- Implemented sign-in/login using Python–SQL connectivity

LOG-5: 14/07/2025

- Created SQL table Timetable

LOG-6: 21/07/2025

- Created SQL table Leaves
- Worked on Teacher Dashboard: View Timetable option

LOG-7: 28/07/2025

- Implemented Apply Leave interface
- Added backend logic for saving leave (MySQL + fallback)

LOG-8: 04/08/2025

- Created Coordinator Dashboard layout
- Added option to view all timetables

LOG-9: 11/08/2025

- Implemented pending leave view for coordinator

LOG-10: 18/08/2025

- Added leave approval functionality
- Tested updating leave status in database and fallback storage

LOG-11: 25/08/2025

- Implemented substitute teacher assignment logic
- Added random substitute selection feature

LOG-12: 01/09/2025

- Added substitution log storage using CSV
- Implemented “View Substitution Log” screen

LOG-13: 08/09/2025

- Integrated all modules: Login → Teacher Dashboard → Coordinator Dashboard

LOG-14: 15/09/2025

- Added error handling for invalid dates and missing files
- Improved fallback mechanism for timetable and leave retrieval

LOG-15: 22/09/2025

- Enhanced date validation and user prompts (messagebox)
- Ensured data consistency between DB and fallback files

LOG-16: 29/09/2025

- Debugged substitution logic thoroughly
- Ensured correct substitute mapping for all periods

LOG-17: 06/10/2025

- Enhanced GUI layout: alignment, fonts, spacing
- Improved data table readability (timetable, leave list, substitution list)

LOG-18: 13/10/2025

- Final integration of all features into one cohesive program
- Conducted full system testing

LOG-19: 20/10/2025

- Cleaned and optimized code
- Improved database connection error handling
- Added detailed comments for clarity

LOG-20: 27/10/2025

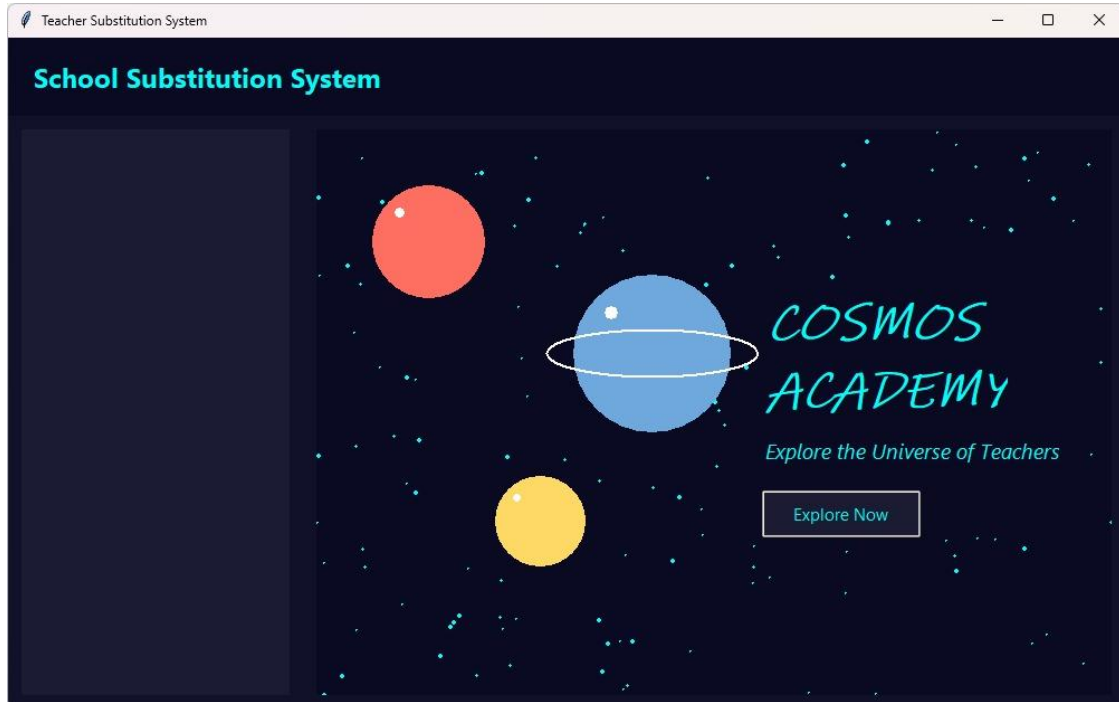
- Polished GUI styling: colours, window sizes, backgrounds
- Improved user-friendliness and readability

LOG-21: 03/11/2025

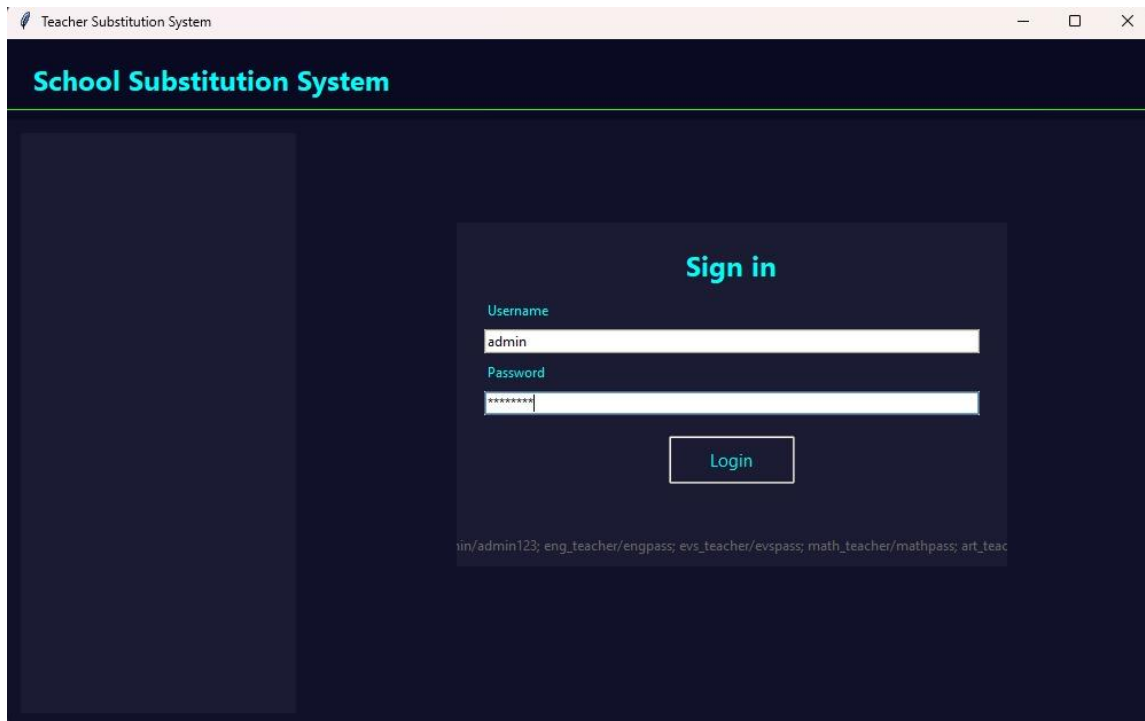
- Final review and polishing
- Completion of project and submission

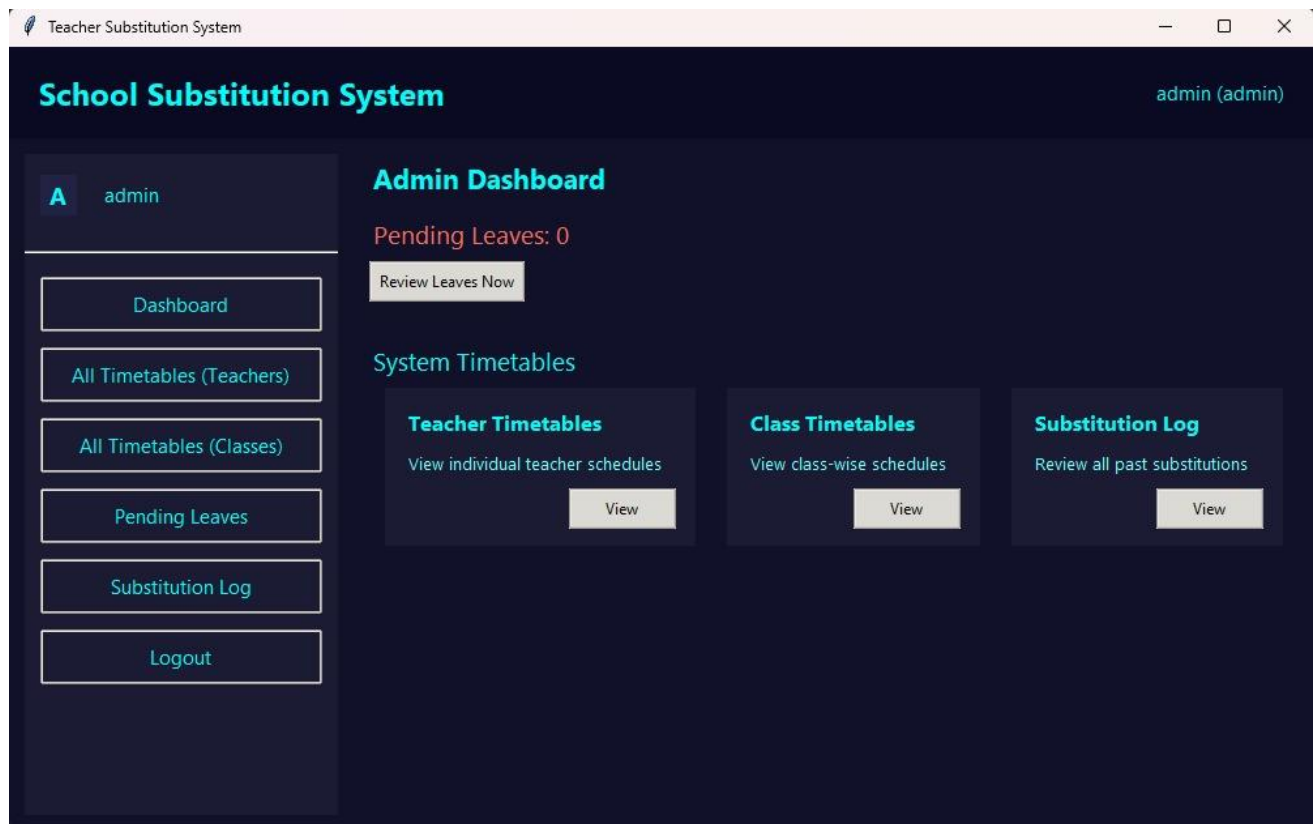
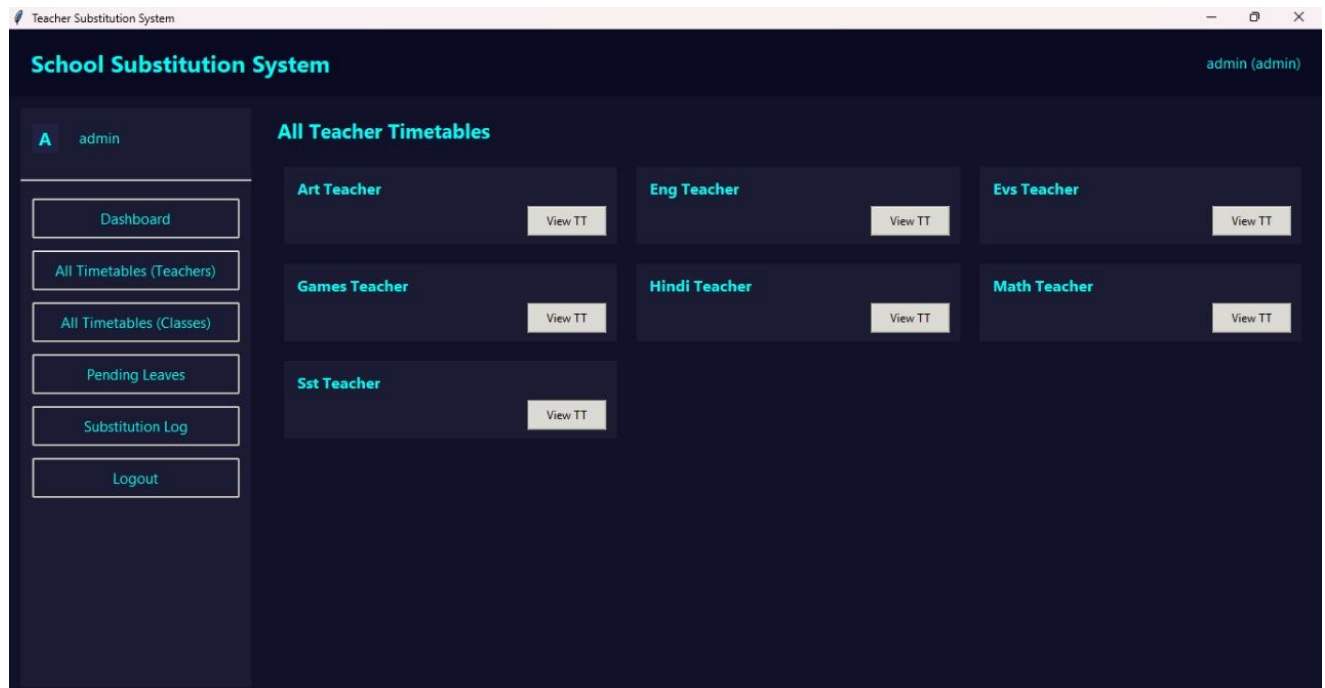
SAMPLE OUTPUT

WELCOME SCREEN:



ADMIN LOGIN SCREEN:



ADMIN WINDOW:**VIEWING ALL THE TEACHER'S TIME TABLES:**

HOW THE TEACHERS TIME TABLE APPEARS :

The screenshot shows the 'School Substitution System' interface. On the left is a sidebar with a user profile 'A admin' and navigation buttons: Dashboard, All Timetables (Teachers), All Timetables (Classes), Pending Leaves, Substitution Log, and Logout. The main content area is titled 'TIMETABLE – MATH TEACHER TIMETABLE' and displays a grid of teacher assignments for Monday through Friday.

DAY	P1	P2	P3	BREAK	P4	P5	P6	LUNCH	P7	P8	P9
Monday	1b	1a	P3	BREAK	P4	1c	P6	LUNCH	1a	P8	1b
Tuesday	P1	1c	1b	BREAK	1a	P5	1a	LUNCH	P7	1b	1a
Wednesday	1a	P2	1c	BREAK	1b	P5	1a	LUNCH	1c	P8	1b
Thursday	P1	1c	1a	BREAK	P4	P5	1b	LUNCH	1a	P8	1c
Friday	P1	1b	P3	BREAK	1a	1c	1a	LUNCH	1b	1c	P9

VIEWING THE TIME TABLE OF ALL THE CLASSES:

The screenshot shows the 'School Substitution System' interface with the 'All Class Timetables' view selected. The sidebar is identical to the previous screenshot. The main content area is titled 'All Class Timetables' and displays three buttons for viewing the timetables for Class 1A, Class 1B, and Class 1C. Each button has a 'View TT' link next to it.

HOW THE CLASS'S TIMETABLE APPEARS:

Teacher Substitution System

School Substitution System admin (admin)

A admin

Dashboard

All Timetables (Teachers)

All Timetables (Classes)

Pending Leaves

Substitution Log

Logout

TIMETABLE – CLASS 1A TIMETABLE

DAY	P1	P2	P3	BREAK	P4	P5	P6	LUNCH	P7	P8	P9
Monday	English	Math	EVS	BREAK	Hindi	Social	English	LUNCH	Math	EVS	Games
Tuesday	Hindi	Social	Games	BREAK	Math	English	Math	LUNCH	Hindi	Social	Math
Wednesday	Math	English	Hindi	BREAK	Social	EVS	Math	LUNCH	EVS	Games	Hindi
Thursday	EVS	Games	Math	BREAK	English	Hindi	Social	LUNCH	Math	Hindi	English
Friday	Social	Hindi	English	BREAK	Math	EVS	Math	LUNCH	English	Games	EVS

APPLYING FOR LEAVE:

Teacher Substitution System

School Substitution System math_teacher (teacher)

M math_teacher

Dashboard

My Timetable

Apply Leave

Substitution Log

Logout

Apply for Leave

Date (DD/MM/YYYY): 25/12/2025

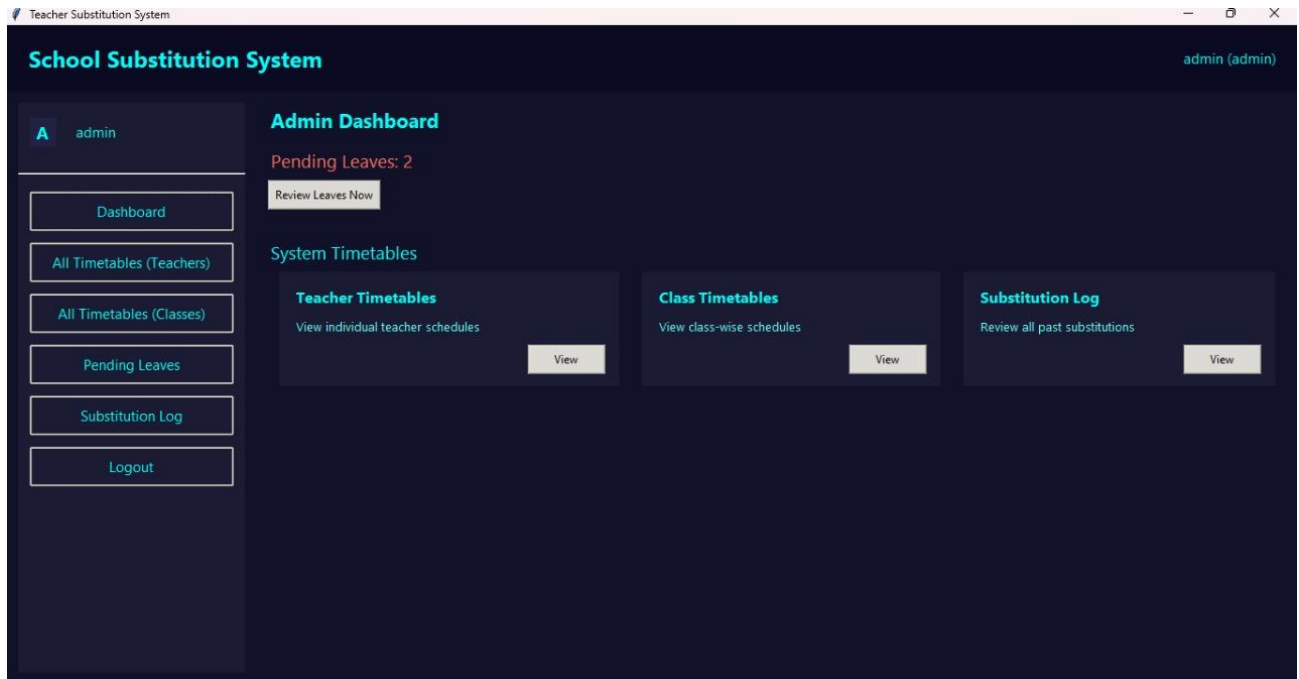
Reason: very very sick :)

Success

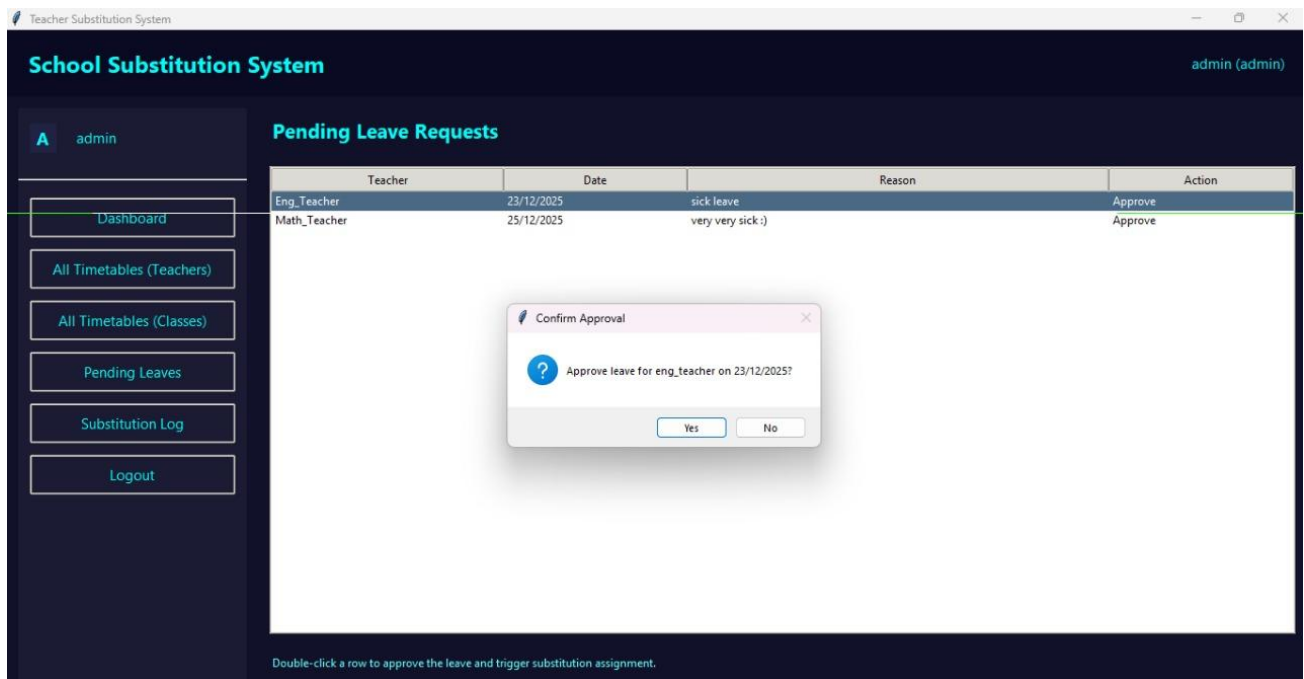
Leave submitted for 25/12/2025. Pending Admin approval.

OK

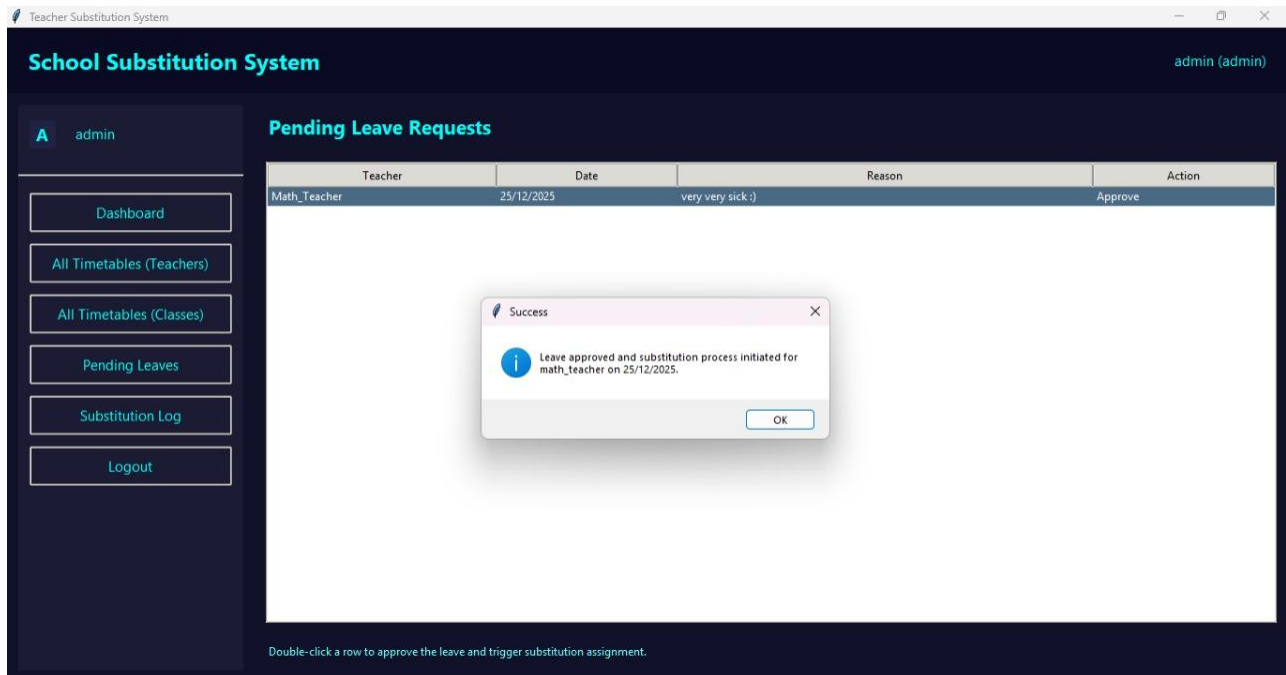
WINDOW SHOWING NUMBER OF PENDING LEAVES TO APPROVE:



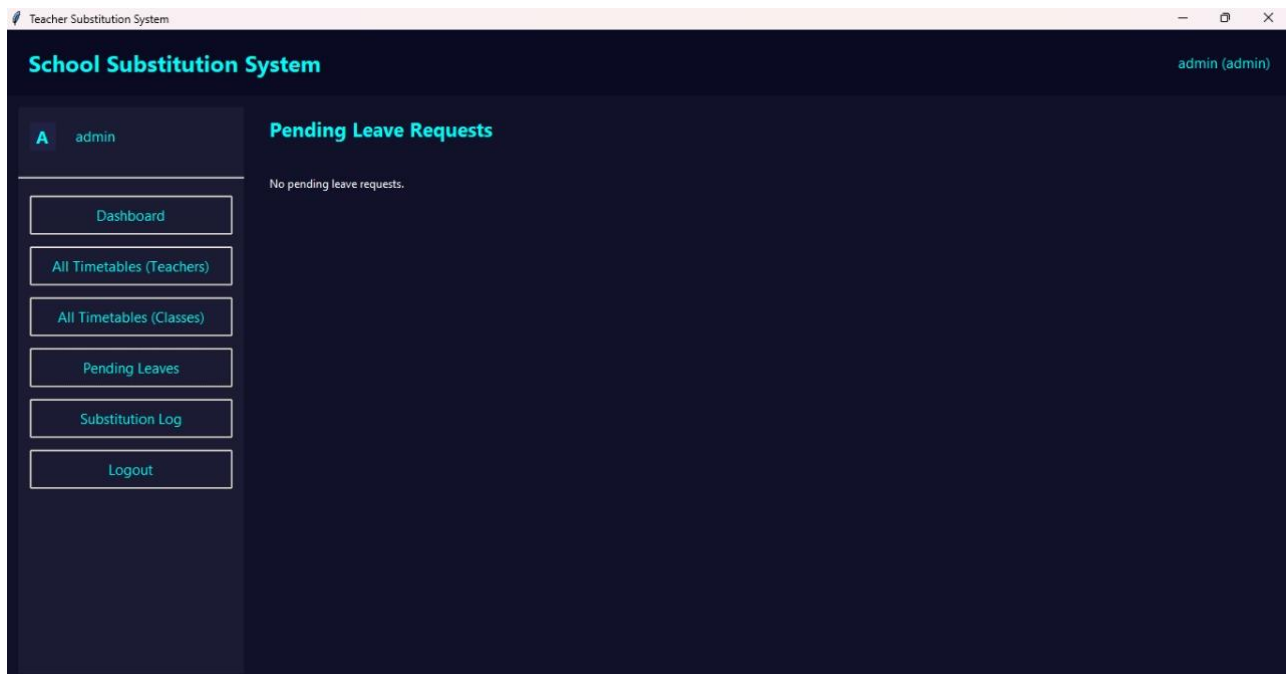
ADMIN IS APPROVING THE LEAVE:



LEAVE HAS BEEN APPROVED AND NEW TEACHERS IS BEING SUBSTITUTED:



WINDOW AFTER APPROVING ALL THE LEAVES:



HOW THE SUBSTITUTION LOG APPEARS:

Teacher Substitution System

School Substitution System admin (admin)

A admin

Dashboard

All Timetables (Teachers)

All Timetables (Classes)

Pending Leaves

Substitution Log

Logout

Substitution Log

Date	Period	Absent Teacher	Substitute Teacher	Delete
09/12/2025	3	Eng_Teacher	Math_Teacher	Delete
09/12/2025	5	Eng_Teacher	Games_Teacher	Delete
09/12/2025	8	Eng_Teacher	Sst_Teacher	Delete
09/12/2025	9	Eng_Teacher	Games_Teacher	Delete
23/12/2025	1	Eng_Teacher	Games_Teacher	Delete
23/12/2025	2	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	3	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	5	Eng_Teacher	Math_Teacher	Delete
23/12/2025	8	Eng_Teacher	Games_Teacher	Delete
23/12/2025	9	Eng_Teacher	Games_Teacher	Delete
25/12/2025	2	Math_Teacher	Evs_Teacher	Delete
25/12/2025	3	Math_Teacher	Sst_Teacher	Delete
25/12/2025	6	Math_Teacher	Hindi_Teacher	Delete
25/12/2025	7	Math_Teacher	Eng_Teacher	Delete
25/12/2025	9	Math_Teacher	Games_Teacher	Delete

Update Substitution:

Date (DD/MM/YYYY): Period:

New Teacher Username:

ADMIN IS TRYING TO DELETE A SUBSTITUTION RECORD:

Teacher Substitution System

School Substitution System admin (admin)

A admin

Dashboard

All Timetables (Teachers)

All Timetables (Classes)

Pending Leaves

Substitution Log

Logout

Substitution Log

Date	Period	Absent Teacher	Substitute Teacher	Delete
09/12/2025	3	Eng_Teacher	Math_Teacher	Delete
09/12/2025	5	Eng_Teacher	Games_Teacher	Delete
09/12/2025	8	Eng_Teacher	Sst_Teacher	Delete
09/12/2025	9	Eng_Teacher	Games_Teacher	Delete
23/12/2025	1	Eng_Teacher	Games_Teacher	Delete
23/12/2025	2	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	3	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	5	Eng_Teacher	Math_Teacher	Delete
23/12/2025	8	Eng_Teacher	Games_Teacher	Delete
23/12/2025	9	Eng_Teacher	Games_Teacher	Delete
25/12/2025	2	Eng_Teacher	Evs_Teacher	Delete
25/12/2025	3	Eng_Teacher	Sst_Teacher	Delete
25/12/2025	6	Eng_Teacher	Hindi_Teacher	Delete
25/12/2025	7	Eng_Teacher	Eng_Teacher	Delete
25/12/2025	9	Eng_Teacher	Games_Teacher	Delete

Confirm Delete

Delete this substitution?

Date: 09/12/2025
Period: 3
Absent: Eng_Teacher
Substitute: Math_Teacher

Update Substitution:

Date (DD/MM/YYYY): Period:

New Teacher Username:

SUCCESSFULLY DELETED A RECORD:

The screenshot shows the 'School Substitution System' interface. On the left is a sidebar with navigation links: Dashboard, All Timetables (Teachers), All Timetables (Classes), Pending Leaves, Substitution Log, and Logout. The main area is titled 'Substitution Log' and displays a table with columns: Date, Period, Absent Teacher, Substitute Teacher, and Delete. A modal dialog titled 'Deleted' is open in the center, displaying an information icon and the message 'Substitution record removed.' with an 'OK' button.

Date	Period	Absent Teacher	Substitute Teacher	Delete
09/12/2025	3	Eng_Teacher	Math_Teacher	Delete
09/12/2025	5	Eng_Teacher	Games_Teacher	Delete
09/12/2025	8	Eng_Teacher	Sst_Teacher	Delete
09/12/2025	9	Eng_Teacher	Games_Teacher	Delete
23/12/2025	1	Eng_Teacher	Games_Teacher	Delete
23/12/2025	2	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	3		Evs_Teacher	Delete
23/12/2025	5		Math_Teacher	Delete
23/12/2025	8		Games_Teacher	Delete
23/12/2025	9		Games_Teacher	Delete
25/12/2025	2		Evs_Teacher	Delete
25/12/2025	3		Sst_Teacher	Delete
25/12/2025	6		Hindi_Teacher	Delete
25/12/2025	7		Eng_Teacher	Delete
25/12/2025	9	Math_Teacher	Games_Teacher	Delete

Below the table is an 'Update Substitution:' form with fields for Date (DD/MM/YYYY), Period, and New Teacher Username, along with an 'Update' button.

TRYING TO UPDATE A RECORD BUT ERROR IS THROWN AS THE TEACHER WHO YOU ARE TRYING TO ALLOT SUBSTITUTION IS NOT FREE IN THAT PARTICULAR PERIOD:

The screenshot shows the 'School Substitution System' interface. The 'Substitution Log' table is visible. A modal dialog titled 'Busy Teacher' is open in the center, displaying an error icon and the message 'games_teacher already has a class.' with an 'OK' button.

Date	Period	Absent Teacher	Substitute Teacher	Delete
09/12/2025	5	Eng_Teacher	Games_Teacher	Delete
09/12/2025	8	Eng_Teacher	Sst_Teacher	Delete
09/12/2025	9	Eng_Teacher	Games_Teacher	Delete
23/12/2025	1	Eng_Teacher	Games_Teacher	Delete
23/12/2025	2	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	3	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	5		Math_Teacher	Delete
23/12/2025	8		Games_Teacher	Delete
23/12/2025	9		Games_Teacher	Delete
25/12/2025	2		Evs_Teacher	Delete
25/12/2025	3		Sst_Teacher	Delete
25/12/2025	7		Eng_Teacher	Delete
25/12/2025	9		Games_Teacher	Delete

The 'Update Substitution:' form at the bottom shows the Date set to 25/12/2025, Period set to 7, and New Teacher Username set to Games_Teacher. The 'Update' button is visible.

AFTER UPDATING A RECORD SUCCESSFULLY:

The screenshot shows the 'School Substitution System' interface with the user 'admin (admin)'. The left sidebar contains navigation links: Dashboard, All Timetables (Teachers), All Timetables (Classes), Pending Leaves, Substitution Log, and Logout. The main area displays the 'Substitution Log' table, which lists substitution records. A modal window titled 'Success' is displayed in the center, indicating 'Substitution updated.' with an 'OK' button.

Date	Period	Absent Teacher	Substitute Teacher	Delete
09/12/2025	5	Eng_Teacher	Games_Teacher	Delete
09/12/2025	8	Eng_Teacher	Sst_Teacher	Delete
09/12/2025	9	Eng_Teacher	Games_Teacher	Delete
23/12/2025	1	Eng_Teacher	Games_Teacher	Delete
23/12/2025	2	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	3	Eng_Teacher	Evs_Teacher	Delete
23/12/2025	5	Eng_Teacher	Math_Teacher	Delete
23/12/2025	8	Eng_Teacher	Games_Teacher	Delete
23/12/2025	9	Eng_Teacher	Games_Teacher	Delete
25/12/2025	2	Eng_Teacher	Evs_Teacher	Delete
25/12/2025	3	Eng_Teacher	Sst_Teacher	Delete
25/12/2025	6	Eng_Teacher	Hindi_Teacher	Delete
25/12/2025	7	Eng_Teacher	Eng_Teacher	Delete
25/12/2025	9	Eng_Teacher	Games_Teacher	Delete

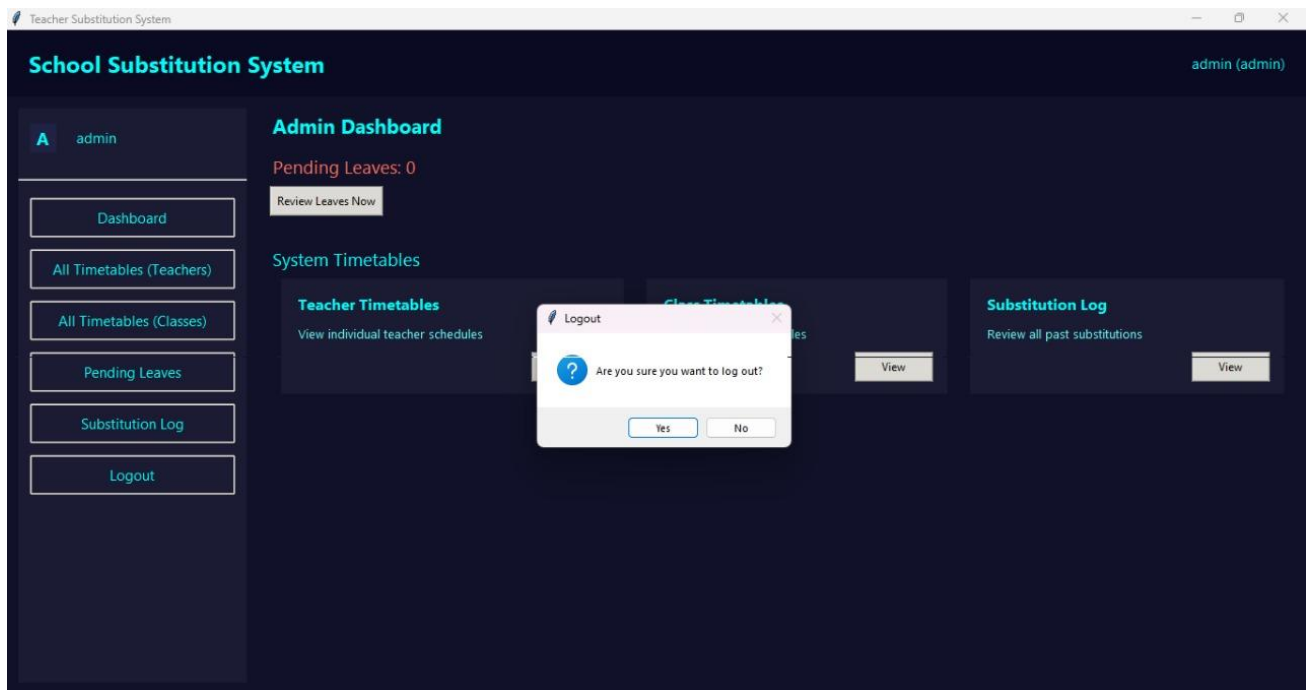
Below the table, there is an 'Update Substitution:' section with input fields for Date (DD/MM/YYYY), Period, and New Teacher Username, along with an 'Update' button.

HOW THE UPDATED RECORD IS HIGHLIGHTED WHEN THE TEACHER VIEWS IT :

The screenshot shows the 'School Substitution System' interface with the user 'math_teacher (teacher)'. The left sidebar contains navigation links: Dashboard, My Timetable, Apply Leave, Substitution Log, and Logout. The main area displays the 'Substitution Log' table, which lists substitution records. The record for 23/12/2025, Period 5, where Eng_Teacher is absent and Math_Teacher is the substitute, is highlighted in blue.

Date	Period	Absent Teacher	Substitute Teacher
23/12/2025	5	Eng_Teacher	Math_Teacher
25/12/2025	2	Math_Teacher	Evs_Teacher
25/12/2025	3	Math_Teacher	Sst_Teacher
25/12/2025	6	Math_Teacher	Hindi_Teacher
25/12/2025	7	Math_Teacher	Eng_Teacher
25/12/2025	9	Math_Teacher	Games_Teacher

THE ADMIN IS TRYING TO LOG OUT:

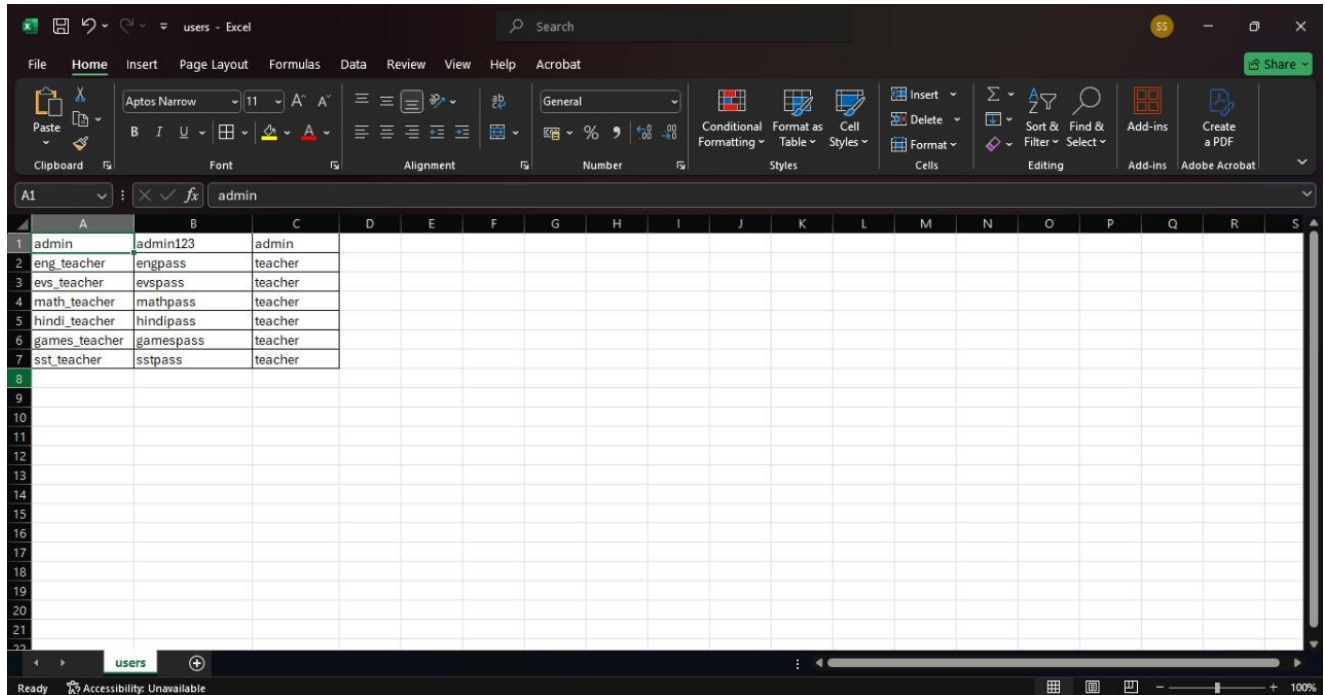


THE SUBSTITUTION LOG IN THE FORM OF CSV FILE:

The screenshot shows an Excel spreadsheet titled 'substitution_log - Excel'. The data is organized in a table with the following columns: Date, Period, Absent, and Substitute.

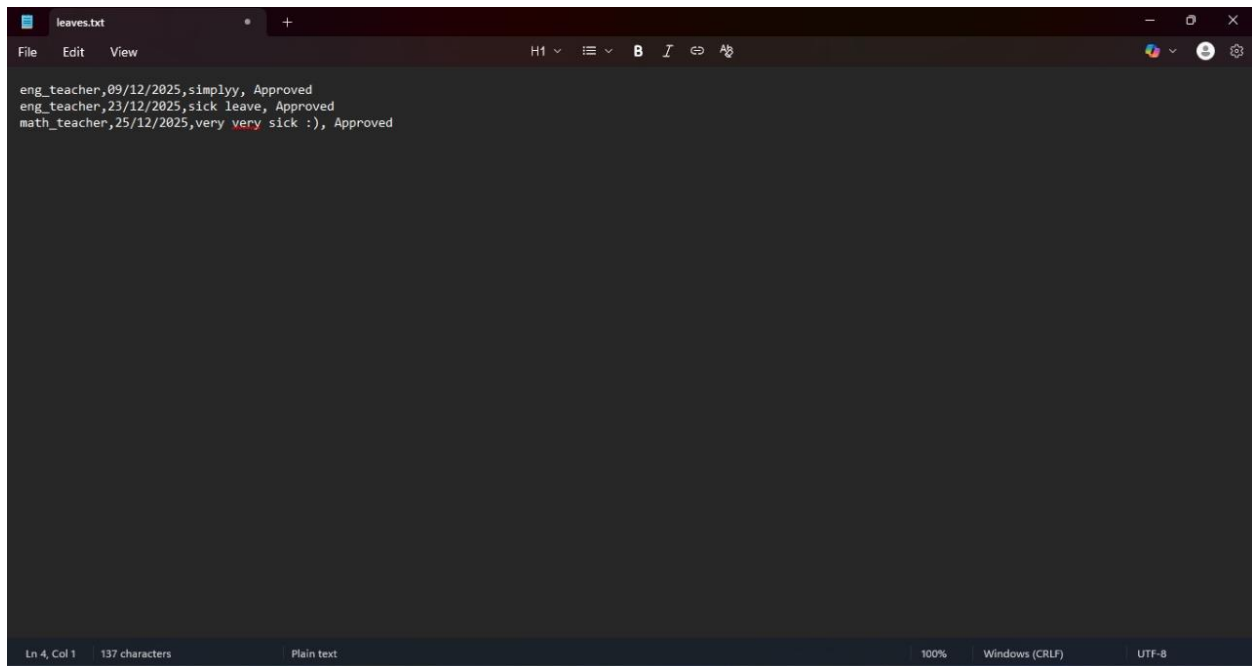
Date	Period	Absent	Substitute
09-12-2025	5	eng_teacher	games_teacher
09-12-2025	8	eng_teacher	sst_teacher
09-12-2025	9	eng_teacher	games_teacher
23-12-2025	1	eng_teacher	games_teacher
23-12-2025	2	eng_teacher	evs_teacher
23-12-2025	3	eng_teacher	evs_teacher
23-12-2025	5	eng_teacher	math_teacher
23-12-2025	8	eng_teacher	games_teacher
23-12-2025	9	eng_teacher	games_teacher
25-12-2025	2	math_teacher	evs_teacher
25-12-2025	3	math_teacher	sst_teacher
25-12-2025	6	math_teacher	hindi_teacher
25-12-2025	7	math_teacher	eng_teacher
25-12-2025	9	math_teacher	games_teacher

THE LOGIN DETAILS STORED IN THE CSV FILE:



	A	B	C
1	admin	admin123	admin
2	eng_teacher	engpass	teacher
3	evs_teacher	evspass	teacher
4	math_teacher	mathpass	teacher
5	hindi_teacher	hindipass	teacher
6	games_teacher	gamespass	teacher
7	sst_teacher	sstpass	teacher
8			
9			
10			
11			
12			
13			
14			
15			
16			
17			
18			
19			
20			
21			
22			

THE LEAVE RECORDS WHICH ARE STORED IN TEXT FILE:



```
eng_teacher,09/12/2025,simplyy, Approved
eng_teacher,23/12/2025,sick leave, Approved
math_teacher,25/12/2025,very very sick :), Approved
```

BIBLIOGRAPHY

- <https://www.tutorialspoint.com/python/>
- <https://docs.python.org/3/library/tkinter.html>
- <https://www.geeksforgeeks.org/python-mysql-connector/>
- <http://stackoverflow.com/>
- <https://www.mysql.com/>
- <https://pythonexamples.org/python-tkinter-treetview/>
- <https://realpython.com/python-datetime/>
- https://www.w3schools.com/python/python_csv.asp
- <https://pythonbasics.org/python-random/>
- <https://effbot.org/>

